TEACHING & MENTORING

Teaching Portfolio

Raj Patel · PhD Student, Department of Computer Science, The University of Alabama · rpatel38@ua.edu · rajpatel-ua.github.io

How I learned to teach, what I believe about teaching, and how I would teach a course at The University of Alabama: **CS 456, AI for Cyber Security**.

01 AUTOBIOGRAPHY

My teaching autobiography

Three starter questions about where my teaching comes from, and how I'll keep developing my pedagogical content knowledge (PCK): knowing not just the material, but how to teach it.

Q1 How did you learn to teach?

Mostly by watching good teachers and paying attention to what worked for me as a student. The professors I learned the most from grounded every idea in a real example. One showed us how soft computing was used in Japan's railway systems, and I remembered the method because I remembered where it ran. Real-world examples helped me retain what I learned and gave me a foothold for learning the next thing.

I also learned by training people. In six years as a systems administrator, I supervised and trained student workers, wrote the documentation they learned from, and supported faculty and staff through new systems. That taught me that a clear explanation starts from what the learner already knows.

Q2 How did this help develop your PCK?

It taught me to build a bank of several examples for every concept, so that if one doesn't land, another will. In security and AI, the best examples are real incidents and real data, and I know many of them first hand from running production systems and from my research.

It also taught me to start from the learning outcomes and work backward. Every slide, example and lab should serve an outcome, and the only proof that it does is evidence that students are learning. In CS 456, for example, my research showed me a pitfall common in this field: models that look excellent because training and test data overlap. So I plan the lesson that exposes data leakage before students build their own detectors.

Q3

How can you continue to develop your PCK?

By treating every class as data. I'll read students' comments and minute papers for what they actually retained, not just what they enjoyed. I'll ask faculty colleagues which techniques and technologies keep their classrooms engaging and interactive, and invite them to observe mine. And I'll try new ways of evaluating what students know, keeping the ones that show results. Coursework such as AHE 603, College and University Teaching, gives me the vocabulary and tools to do this deliberately.

02 OTHER AREAS

My teaching toolkit

The people, publications, events and resources I turn to as I grow as a teacher.

PEOPLE

Faculty I turn to about teaching CS

- Dr. Shahram Rahimi
- Dr. Mahmoud Mahmoud
- Dr. Charan Gudla

JOURNALS

Teaching scholarship I read

- *College Teaching*
- *Journal on Excellence in College Teaching*
- *Journal of the Scholarship of Teaching and Learning*

CONFERENCES

Where teachers meet

- Lilly Conference on College and University Teaching
- IEEE Frontiers in Education (FIE)
- ACM SIGCSE Technical Symposium

BOOKS

On my shelf

- *McKeachie's Teaching Tips* (Svinicki & McKeachie)
- *The New Science of Learning* (Doyle & Zakrajsek)

ONLINE TOOLS

Resources I use

- K. Patricia Cross Academy (teaching technique videos)
- UA teaching development resources
- Teaching centers at MIT, Harvard and Princeton
- Government resources such as the NIST NICE Framework
- Generative AI (e.g., ChatGPT) for drafting practice questions, checked by me

PREPARATION & MENTORING

Experience so far

- AHE 603, College and University Teaching, UA (Fall 2026)
- Supervised and trained four student workers at Mississippi State (2019–2025), improving team efficiency by 40% through documentation

Teaching philosophy statement

Built on five sentence starters, each grounded in the course below.

I did not plan to become a computer scientist. I enrolled as a Mechanical Engineering student, took a programming course from a professor who made the material both clear and fun, and changed my major. That experience, along with six years of running real systems as a university systems administrator, shapes how I teach. I describe my approach below through a course I have designed for The University of Alabama: CS 456, AI for Cyber Security. In it, students use machine learning to solve security problems and then learn how attackers exploit those same models.

I believe students learn best when they have a solid foundation, enjoy the work, and then have room to explore on their own. Structure comes first, because students can't explore what they can't yet reason about. Once they have that foundation, curiosity does more than any directive. Every module in CS 456 follows this arc. In the first lab, students build a network intrusion detector by following a guided notebook through feature extraction, training and evaluation. The second half of the lab has no instructions, only a challenge: find a way to make your detector look better than it really is, then explain why. Students enjoy that open-ended part, and the enjoyment matters. The same mix of clear foundations and freedom to tinker is what pulled me from mechanical engineering into computing.

As an educator, my role is to provide the tools, knowledge and motivation students need, and to model how a practitioner thinks, including how a practitioner fails. The lessons I learned the hard way are the ones I still remember, so I share mine. When we discuss patch management, I describe patching hundreds of university machines against live vulnerability reports, and why "just update everything" is harder than it sounds. When we discuss peer review, I show students how my first-author survey on securing machine-learning pipelines came back from four reviewers with a major-revision decision, and how working through every comment turned it into the first manuscript accepted at ACM Transactions on AI Security and Privacy. Students see that setbacks are part of the work, not a verdict on their ability. My role also includes building an environment where it is safe to fail. Drawing on my experience deploying an OpenStack teaching cloud and virtual desktops for 300 concurrent users, I give each student an isolated sandbox where attacking a model can't harm anyone. Every offensive exercise is covered by an acceptable-use agreement that we discuss and sign in the first week.

One way I connect course content to students' prior knowledge or experience is by starting from what they already know, then showing how the content works in real systems, backed by evidence. Every student has received a phishing email, reset a password or seen an antivirus warning. I begin there and build toward the machine-learning systems that now sit behind those experiences. Then I anchor each concept in a documented incident, the way one of my own professors made soft computing unforgettable by showing how it was used in Japan's railway systems. When we study evasion, we read how researchers slipped malware past an AI-based detector. When we study poisoning, we examine how systematically submitting borderline samples to a shared scanning service shifted the decision boundaries of commercial malware classifiers. When we study the machine-learning supply chain, we trace the PyTorch dependency-confusion attack and the ShadowRay

campaign, which used an unauthenticated API to compromise more than five thousand servers. Numbers make the stakes concrete. A detector with a 1% false-positive rate, facing one million network flows a day, sends analysts ten thousand false alarms. Students who have never worked in a security operations center immediately understand why accuracy alone is misleading.

I know meaningful learning has happened when students ask insightful follow-up questions that go beyond what I taught, because those questions show they can use the idea on their own. In one CS 456 lesson, I present a classifier that scores 99% accuracy on a popular public dataset. Students then split the same data by time instead of at random and watch performance fall. A student who asks "what's the right way to split?" has followed the lesson. A student who asks "could an attacker time an attack to exploit the drift between our training data and today's traffic?" has learned it, because they connected evaluation, deployment and the adversary on their own. This lesson grows directly out of my research. In benchmarking spiking neural networks for intrusion detection, one of the clearest lessons was how much reported results depend on keeping training and test data properly separated. I design assessments to draw out this kind of transfer. The final project asks teams to threat-model a real AI pipeline with MITRE ATLAS, and I grade the questions they raise about their own design as seriously as their answers.

I will continue to develop my teaching by gathering meaningful feedback from students and acting on it where they can see the results. Each week ends with a one-minute paper asking for the "muddiest point" of the lesson, and I open the next class by addressing the most common answers. At mid-semester I run an anonymous start-stop-continue survey and report back on what I will change. I learned the value of anonymous feedback as a student. One of my professors had classmates anonymously evaluate each in-class presentation, and those comments showed me weaknesses in my presenting that I couldn't see on my own. I will invite faculty colleagues to observe my classes, borrow classroom assessment techniques from the K. Patricia Cross Academy, and keep reading research on how people learn, starting with *McKeachie's Teaching Tips* and Doyle and Zakrajsek's *The New Science of Learning*. I am building that foundation now through UA's College and University Teaching course. Teaching, like security, is never finished: threats change, tools change and students change, so my practice has to keep adapting with them.

Raj Patel

CS 456: AI for Cyber Security

A course I am prepared to teach. It is an existing UA course (catalog) that "explores the intersection of artificial intelligence (AI) and cybersecurity," covering how AI and machine learning solve security problems, how attackers exploit AI's weaknesses, threat intelligence, and malware analysis and classification, with hands-on projects on real datasets.

CREDITS	PREREQUISITE	AUDIENCE
3 credit hours	**CS 465 Artificial Intelligence**	**Upper-level CS and cyber security majors**
FORMAT	LAB ENVIRONMENT	TOOLS
Two 75-minute sessions a week, 15 weeks	**Per-student sandboxed virtual machines**	**Python, scikit-learn, PyTorch, Adversarial Robustness Toolbox**

COURSE GOAL

Prepare students to use machine learning to defend real systems, and to recognize, test and defend against attacks on machine learning itself.

OBJECTIVE 1

Give students hands-on experience applying machine learning to core security problems (intrusion detection, malware classification and threat intelligence) on real datasets.

OBJECTIVE 2

Show students how attackers exploit AI systems across their lifecycle, and give them a safe, authorized environment to practice both attacking and defending models.

OUTCOME 1 · SUPPORTS OBJECTIVE 1

Students will be able to **build an ML-based security detector and evaluate it rigorously**, using leakage-resistant data splits and a false-alert budget, and explain any gap between reported and realistic performance.

Measured by Labs 1–2 and the midterm. Target: 80% of students reach "proficient" on the Lab 1 evaluation rubric.

OUTCOME 2 · SUPPORTS OBJECTIVE 2

Students will be able to **attack and harden an ML model in a sandbox, and threat-model an AI system** across its lifecycle with MITRE ATLAS, communicating risks and mitigations to technical and non-technical audiences.

Measured by Lab 3, the team project and peer review. Target: 80% of teams reach "proficient" on the project rubric.

Course map

WEEKS	MODULE	TOPICS	HANDS-ON WORK	OUTCOME
1–2	Foundations	Threat models, the ML pipeline, MITRE ATT&CK and ATLAS, authorization and ethics	Lab 0: sandbox setup and acceptable-use agreement	1 & 2
3–5	ML for intrusion detection	Flow features, public IDS datasets, class imbalance, false-alert budgets, data leakage	Lab 1: build a detector, then break your own evaluation	1
6–7	Malware analysis	Static and dynamic features, feature-vector datasets, interpreting classifiers	Lab 2: malware family classifier from feature vectors	1
8	Threat intelligence	Mining threat reports, mapping text to ATT&CK techniques with NLP and LLMs	Midterm exam	1
9–11	Adversarial ML	Evasion, poisoning, backdoors, model extraction, documented incidents	Lab 3: red-team the classifier, then harden it	2
12–13	Securing the AI pipeline	The secure MLOps lifecycle, model supply chain, prompt injection, LLM guardrails	In class: threat-model an MLOps pipeline	2
14–15	AI agents in the SOC	Human oversight, auditability, responsible deployment	Final project presentations	2

Assessment

COMPONENT	WEIGHT	WHY IT'S THERE	OUTCOME
Labs (4)	35%	Hands-on skill; each lab ends with a short written reflection	1 & 2
Team project	30%	Proposal, checkpoint, final report and presentation	2
Midterm exam	15%	Individual check on core concepts	1
Minute papers & in-class work	10%	Low-stakes retrieval practice, graded for completion	1 & 2
Peer review	10%	Anonymous, rubric-based feedback on classmates' presentations	2

05 SIGNATURE ASSIGNMENTS

Where research meets the classroom

Each assignment comes straight out of a research question I've worked on.

LAB 1 · OUTCOME 1

Build it, then break your evaluation

Students train an intrusion detector with a random split, then re-evaluate with time- and host-based splits under a fixed false-alert budget. A one-page memo explains the gap.

From my leakage-resistant SNN benchmark (IEEE CogMI 2026)

LAB 3 · OUTCOME 2

Red-team the classifier

Students attack their own Lab 1 detector with evasion and poisoning inside the sandbox, then harden it with adversarial training and measure what the defense costs on clean data.

From my adversarial ML coursework and the Secure MLOps mitigation taxonomy

TEAM PROJECT · OUTCOME 2

Threat-model a real AI pipeline

Teams pick an open-source ML system, map its attack surface across design and setup, development and operations, and propose prioritized mitigations with MITRE ATLAS.

From my Secure MLOps survey (ACM TAISAP 2026)

06 SAMPLE LESSON

"Why your 99% accurate detector is lying"

Week 4, session 2 (75 minutes), supporting Outcome 1. Students will be able to (1) explain why accuracy misleads when attacks are rare, (2) name three ways data can leak between training and test sets, and (3) choose a detection threshold under a false-alert budget. Before class, students read a short piece on base rates and take a five-question warm-up quiz.

MINUTES	ACTIVITY	WHAT STUDENTS DO
0–5	Hook	Live poll: a detector scores 99% accuracy. Would you deploy it?
5–20	Mini-lecture	Base rates and false-alert math: 1% false positives × 1 million flows a day = 10,000 false alarms
20–40	Think-pair-share + notebook	Re-split the same dataset by time and by host, and record what happens to performance
40–55	Case discussion	Groups diagnose what leaked: duplicate records, overlapping time windows, or features that give away the label
55–70	Team decision	Set a false-alert budget for a fictional SOC, pick a threshold, and defend it in one minute
70–75	Minute paper	Write the muddiest point and one question you still have

The lesson feeds directly into Lab 1, and the minute-paper answers shape how the next class opens.

09 PRACTICE

Inclusive and ethical practice

- **Many paths into computing.** I started at a community college and changed majors, so I design for transfer students and career changers. Optional refresher notebooks fill gaps in ML background, and office hours

run as open lab time.

- **More than one way in.** Every lab has a written guide and a recorded walkthrough. Team roles (attacker, defender, reporter) rotate so everyone practices each skill.
- **Authorized, contained practice.** Offensive techniques happen only in isolated sandboxes under a signed acceptable-use agreement. We discuss authorization, responsible disclosure and the law, and point students to CS 247, Cyber Law and Ethics.
- **Accessible by design.** Captioned videos with transcripts, screen-reader-friendly slides and tagged PDFs, alt text on every figure, and color-safe charts. Labs run in virtual machines reached from any browser, so no special hardware is needed. Students registered with UA's Office of Disability Services receive their accommodations promptly, and the syllabus explains how to request them.
- **Transparent grading.** Rubrics are published with each assignment, and frequent low-stakes work reduces the weight of any single exam.
- **Visible feedback loops.** Weekly minute papers and a mid-semester survey, with a "you said, I changed" summary shared back with the class.